



THE UNIVERSITY OF
MELBOURNE

COMP90050 Advanced Database Systems

Semester 1, 2026

Lecturer: Farhana Choudhury (PhD)

Live lecture – Week 7





**Continuing from previous
week's lecture**



Concurrency problem

- What are the possible conflicts/dependencies
- Given a set of concurrent transactions, can we/DBMS determine whether there will be any conflict or not?
- Can conflicts be avoided (without making any change to the intended final output/final state of the database)? - **Yes, locks – as that ensures there's no wormhole!**

This week's lecture contents

- Advanced locking techniques
- Activities



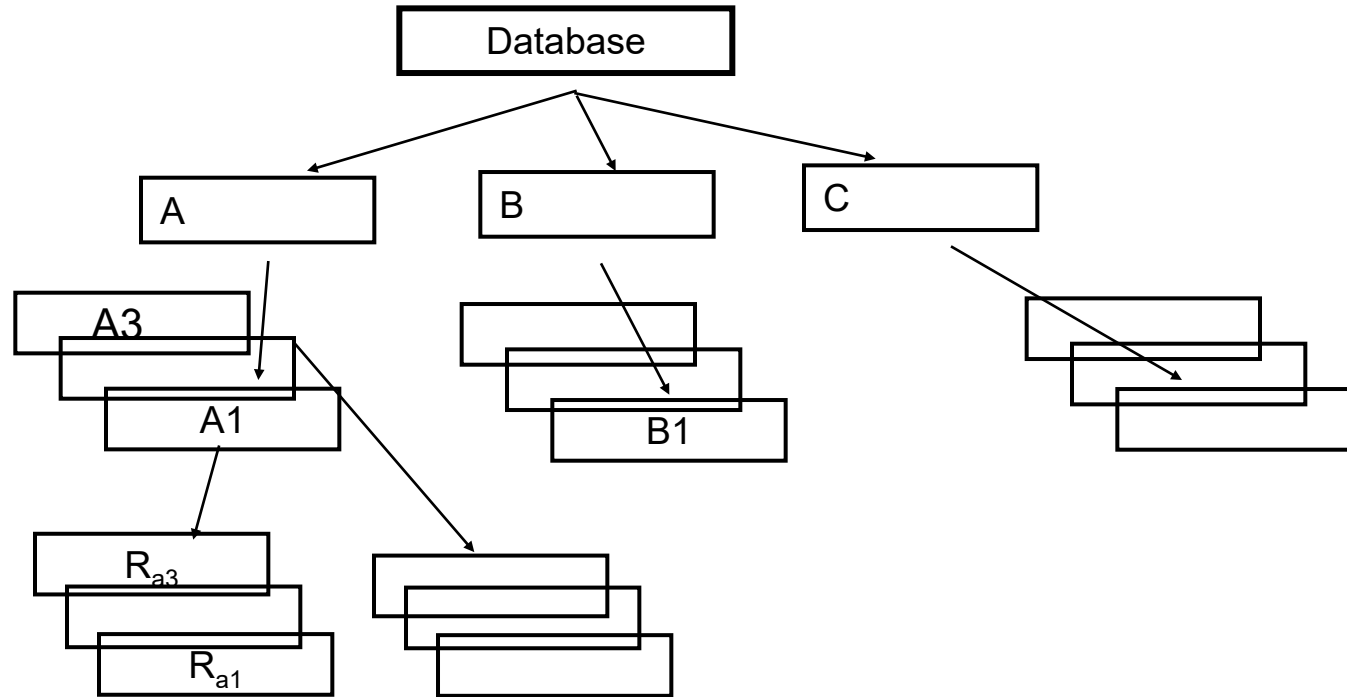
Concurrent transactions – Conflicts and performance issues

Multiple concurrently running transactions may cause conflicts

- Still we try to allow concurrent runs as much as possible for a better performance, while avoiding conflicts as much as possible

Solutions:

1. Use granular locks



Rules:

Lock from root to leaf

If locks are compatible, multiple transactions run concurrently

To set X or S below, get IX or IS above respectively (or higher mode)

T1 needs to read record R_{a1} : T1 needs to lock DB, node A, and A1 in IS mode (or higher mode). Finally, it needs to lock R_{a1} in S mode.

T2 needs to modify record R_{a3} : It can do so after locking DB, node A, and A1 in IX mode. Finally, it needs to lock the R_{a3} in X mode.

Compatibility Mode of Granular Locks

Current	None	IS	IX	S	SIX	U	X
Request	+ - (Next mode) + granted / - delayed						
IS	+(IS)	+(IS)	+(IX)	+(S)	+(SIX)	-(U)	-(X)
IX	+(IX)	+(IX)	+(IX)	-(S)	-(SIX)	-(U)	-(X)
S	+(S)	+(S)	-(IX)	+(S)	-(SIX)	-(U)	-(X)
SIX	+(SIX)	+(SIX)	-(IX)	-(S)	-(SIX)	-(U)	-(X)
U	+(U)	+(U)	-(IX)	+(U)	-(SIX)	-(U)	-(X)
X	+(X)	-(IS)	-(IX)	-(S)	-(SIX)	-(U)	-(X)



Concurrent transactions – Conflicts and performance issues

Discussion:

Why granular lock? Why not take lock for each individual record when needed to read/write?

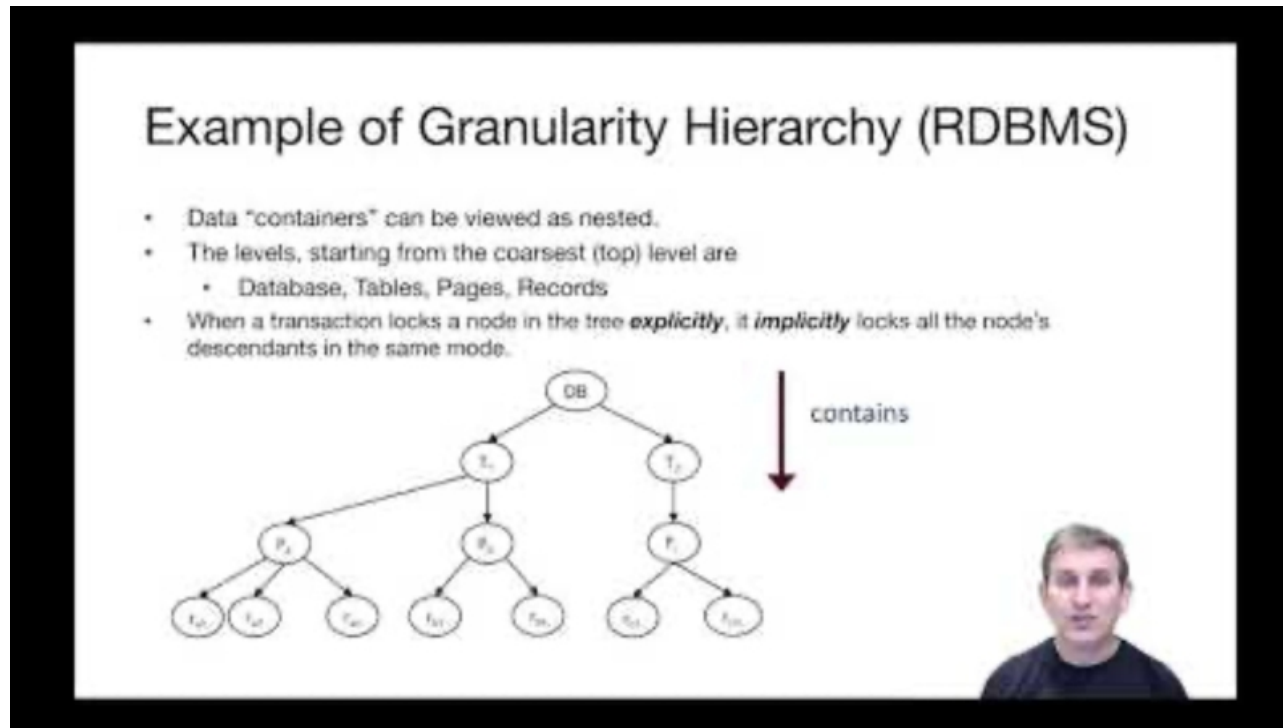
- What will happen if a transaction needs to read a table of millions of records?

Concurrent transactions – Conflicts and performance issues

Discussion:

Why granular lock? Why not take lock for each individual record when needed to read/write?

- What will happen if a transaction needs to read a table of millions of records?





Compatibility Mode of Granular Locks

Current	None	IS	IX	S	SIX	U	X
Request	+ - (Next mode) + granted / - delayed						
IS	+(IS)	+(IS)	+(IX)	+(S)	+(SIX)	-(U)	-(X)
IX	+(IX)	+(IX)	+(IX)	-(S)	-(SIX)	-(U)	-(X)
S	+(S)	+(S)	-(IX)	+(S)	-(SIX)	-(U)	-(X)
SIX	+(SIX)	+(SIX)	-(IX)	-(S)	-(SIX)	-(U)	-(X)
U	+(U)	+(U)	-(IX)	+(U)	-(SIX)	-(U)	-(X)
X	+(X)	-(IS)	-(IX)	-(S)	-(SIX)	-(U)	-(X)

Why Update locks are necessary?

```
T1:
SLock A
Read A
If (A== 3)
{
  % Upgrading Slock to Xlock
    Xlock A
    Write A
}
Unlock A
```

```
T2:
SLock A
Read A
If (A==3)
{
  % Upgrading Slock to Xlock
    Xlock A
    Write A
}
Unlock A
```

```
T3:
SLock A
Read A
Unlock A
```

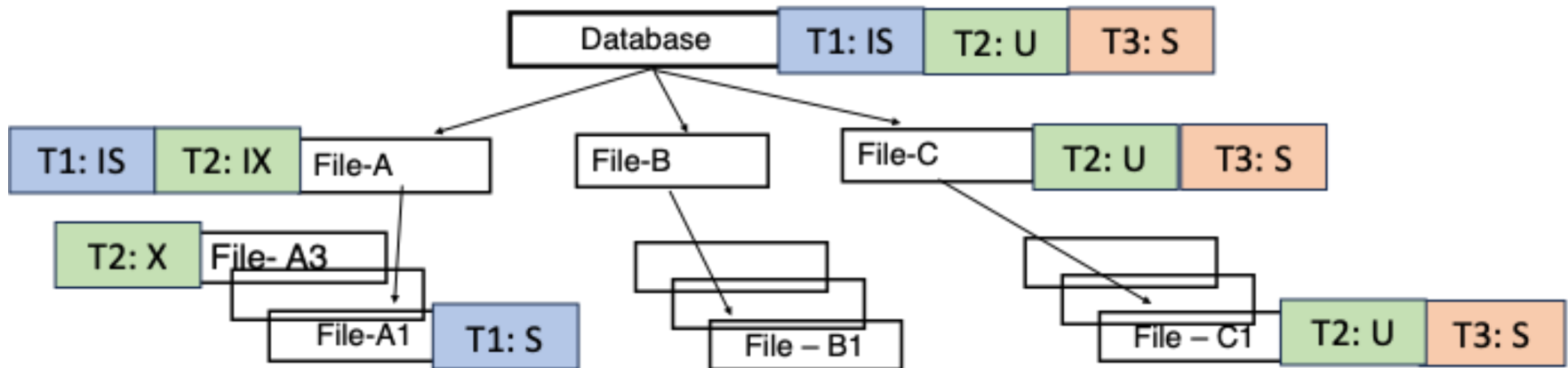
Without update locks, the example transactions can cause very simple deadlock. As per Jim Gray, virtually all deadlocks in System R were found to be of this form!

Discussion: how deadlocks will occur here?

Activity time

Given the hierarchy of database objects and the corresponding granular locks in the following figure, which transactions can run concurrently from the beginning till end (that is, all operations and locks are compatible to run concurrently with another one) and which ones need to be delayed, answer for both scenarios –

- (i) Scenario 1: If the transactions arrive in the order T1-T2-T3. Explain your answer.
- (ii) Scenario 2: If the transactions arrive in the order T1-T3-T2. Explain your answer.





Concurrent transactions – Conflicts and performance issues

Multiple concurrently running transactions may cause conflicts

- Still we try to allow concurrent runs as much as possible for a better performance, while avoiding conflicts as much as possible

Solutions:

1. Use granular locks
2. Optimistic and snapshot isolation



Optimistic locking

When conflicts are rare, transactions can execute operations without managing locks and without waiting for locks - higher throughput

- Use data without locks
- Before committing, each transaction verifies that no other transaction has modified any data that it needs (by taking appropriate locks) – **duration of locks are very short**
- If no conflict, make changes and commit
- If any conflict found, the transaction repeats the attempt



Isolation Concepts ...

Can optimistic locking reduce the chances of deadlock, compared to strict two-phase locking?

Time for a poll - [Pollev.com/farhanachoud585](https://pollev.com/farhanachoud585)





Snapshot locking – more relaxed

When conflicts are rare, transactions can execute operations without managing locks and without waiting for locks - higher throughput

- Use data without locks
- Before committing, each transaction verifies that no other transaction has modified **only** the data that it **needs to write on** (by taking appropriate locks) – **duration of locks are very short**
- If no conflict, make changes and commit
- If any conflict found, the transaction repeats the attempt



Isolation Concepts ...

What kind of applications use relaxed isolation?

- Have you experienced any inconsistency in reading values as users?

Time for a poll - [Pollev.com/farhanachoud585](https://pollev.com/farhanachoud585)

"read-often, write-rarely" scenarios





Isolation Concepts ...

SET TRANSACTION ISOLATION LEVEL {READ UNCOMMITTED |
READ COMMITTED | REPEATABLE READ | SERIALIZABLE}

Slight difference with the four degrees of isolation

- SERIALIZABLE – degree 3
- REPEATABLE READ – like degree 3, but other transactions can insert new rows
- READ COMMITTED – prevents dirty reads like degree 2*
- READ UNCOMMITTED – degree 0

*Options can also be paired with SNAPSHOT on/off

Practicals during tutorials on MySQL server - SET GLOBAL
TRANSACTION ISOLATION LEVEL READ COMMITTED;

Source: <https://docs.microsoft.com/en-us/sql/t-sql/statements/set-transaction-isolation-level-transact-sql?view=sql-server-ver15>



Isolation Concepts ...

SET TRANSACTION ISOLATION LEVEL {READ
UNCOMMITTED | READ COMMITTED | REPEATABLE READ |
SERIALIZABLE}

READ_COMMITTED_SNAPSHOT can be set on or off

If on – Shared locks are not used for reading

- Read committed with READ_COMMITTED_SNAPSHOT off – degree 2
- Read committed with READ_COMMITTED_SNAPSHOT on – *like degree 1



Concurrency and Isolation summary

- Why concurrency control is needed
- Concepts of isolation
 - What causes conflicts
 - Conflict types
- How to avoid conflicts (by locking)
 - Different types of locks
 - Issues caused by locks (deadlocks, convoy)

Core Concepts of Database management system

